

Churn Prediction Pipeline

MLflow Project Summary

Document Type	Technical Architecture & Design Summary
Pipeline Type	End-to-End ML Classification Pipeline
Models Supported	Random Forest · XGBoost
Experiment Tracking	MLflow (local mlruns/)
Language	Python 3.13+

1. Problem Statement

Customer attrition (**churn**) directly impacts a business's revenue and retention metrics. Key challenges include:

- Customer journeys are captured across **multiple time-based snapshots**, making it hard to establish clean train/test splits without data leakage.
 - The business needs to predict churn in **two modes**: binary (Churn vs. Non-Churn) and multiclass (e.g., Churn, Renewal, Expansion, Downsell).
 - Traditional one-off scripts result in **non-reproducible experiments** — hard to compare model runs, hyperparameter sets, or track what produced the best model.
 - Identifying the **top drivers of churn** for downstream customer success intervention is as important as prediction accuracy itself.
-

2. Solution

An automated, modular, and MLflow-tracked end-to-end churn prediction pipeline that:

- Preprocesses trajectory snapshot data into a clean, model-ready format.
 - Splits data **leak-free** by grouping all snapshots of a single customer together.
 - Trains and tunes **Random Forest** (default) or **XGBoost** via GridSearchCV.
 - Evaluates and exports Confusion Matrix, ROC Curve, Classification Report, and Top-20 Feature Importances.
 - Tracks **every experiment run, hyperparameter child run, and final model artifact in MLflow**, enabling full reproducibility and version-controlled model promotion.
-

3. System Design

3a. System Context Diagram

Shows the high-level actors and systems — from data scientist configuration through to downstream deployment.

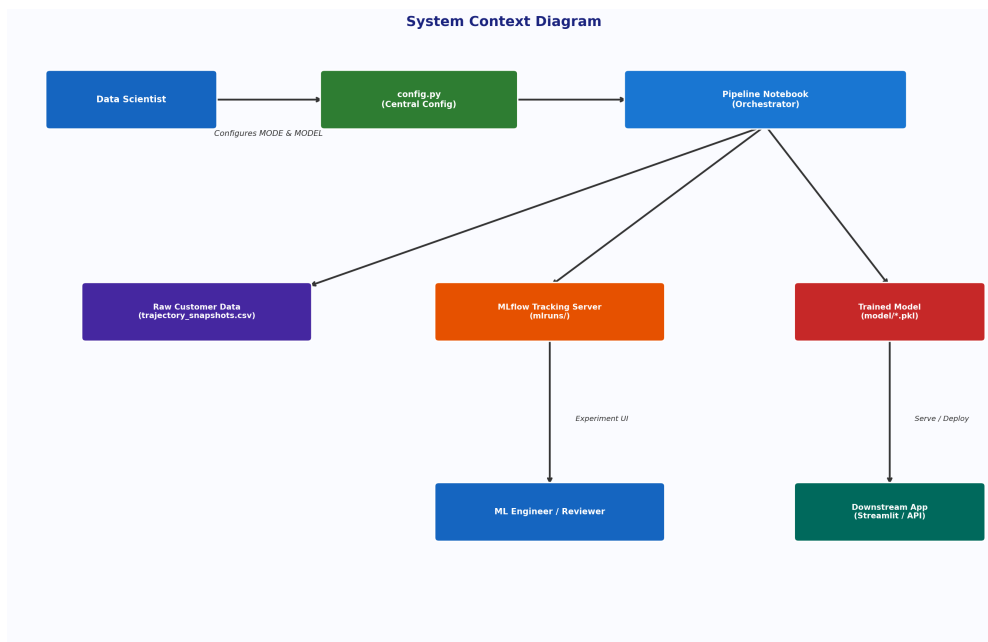


Figure 1 — System Context Diagram

3b. Data Flow Diagram

Illustrates how raw customer snapshots travel through each processing stage — from ingestion to MLflow artifact registration.

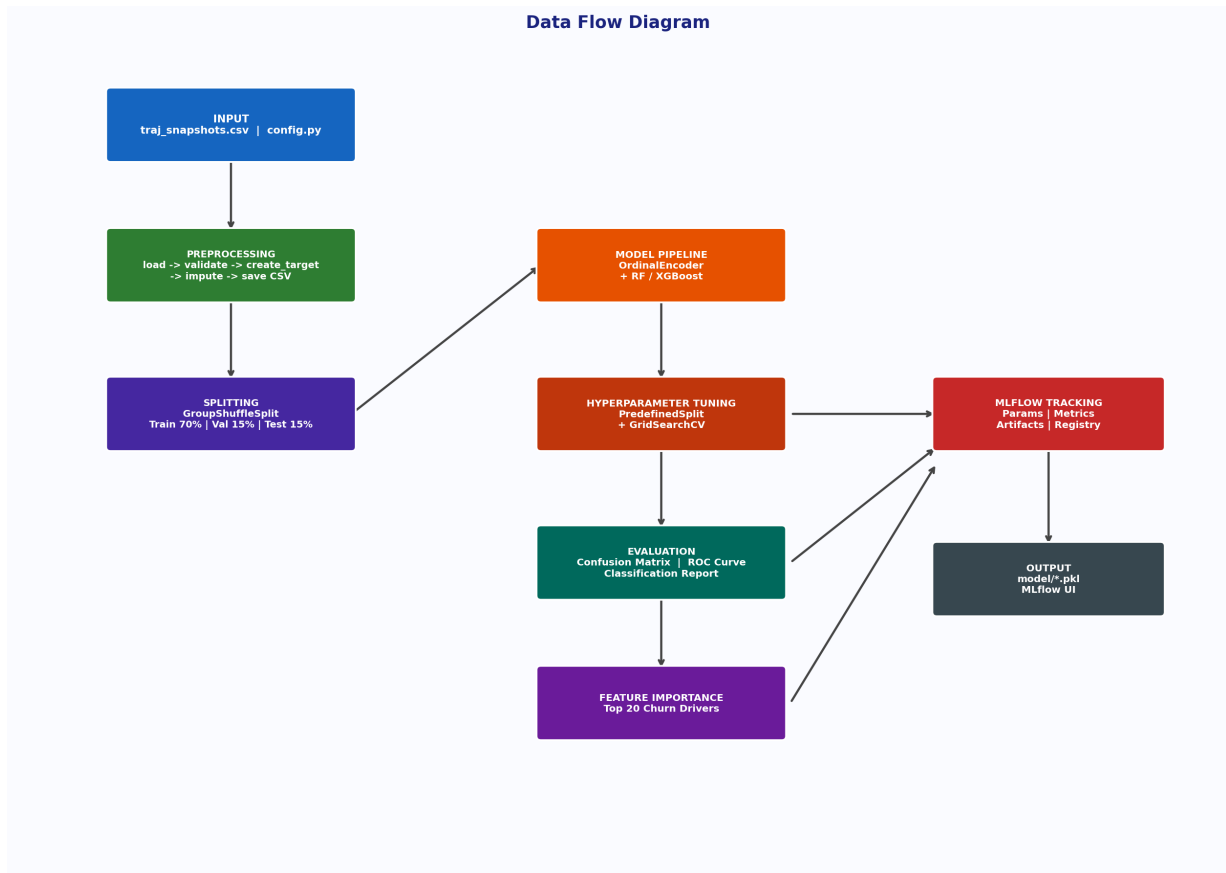


Figure 2 — Data Flow Diagram

3c. Pipeline Orchestrator Activity Diagram

Maps the business-facing pipeline steps to their corresponding Python modules, showing both the logical and technical flow.

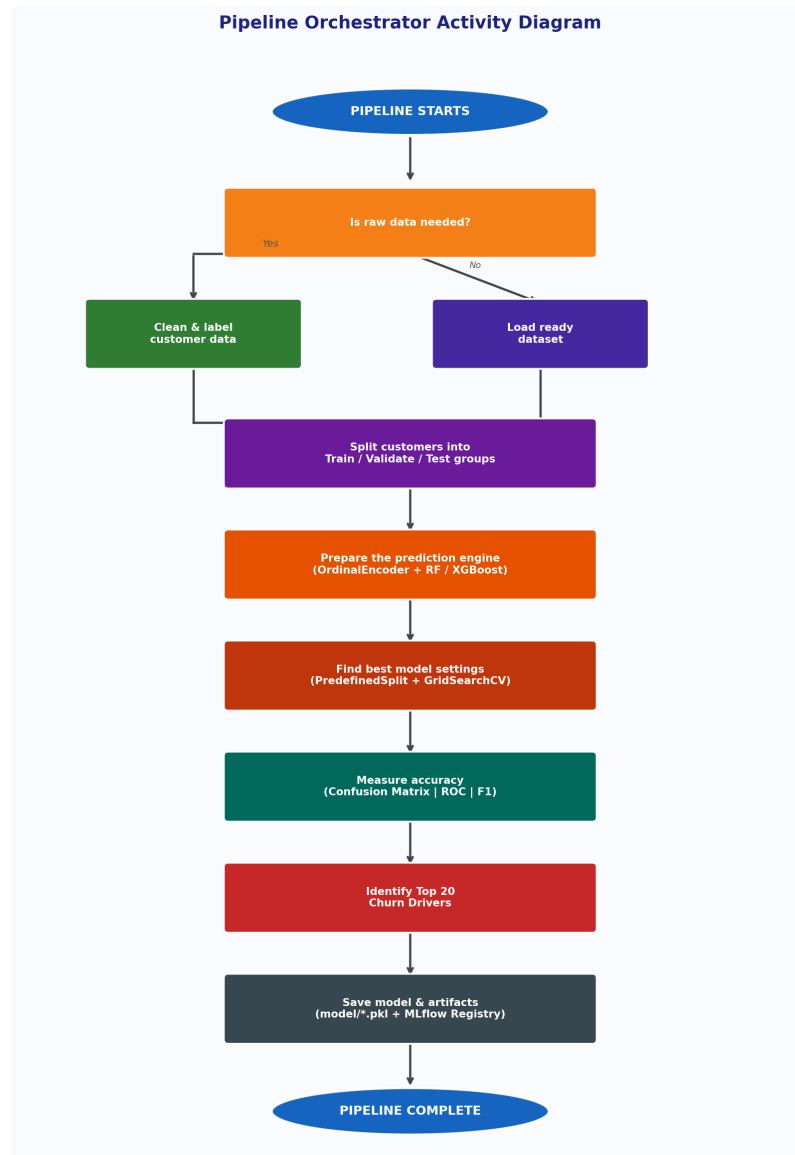


Figure 3 — Pipeline Orchestrator Activity Diagram

4. Technical Stack

Category	Technology	Version
Language	Python	3.13+
Core Data	pandas, numpy, scipy	>=1.3, >=1.20, >=1.7
Machine Learning	scikit-learn	>=1.7.2
Gradient Boosting	XGBoost	>=2.0.0
Experiment Tracking	MLflow	>=2.12.0
Model Serialization	joblib, pickle-mixin	>=1.5.2, >=1.0.2
Visualization	matplotlib, seaborn	>=3.4, >=0.11
Deployment / App	Streamlit	>=1.10.0
Notebook Interface	Jupyter, ipykernel	>=1.0, >=6.0
Stats Modeling	statsmodels	>=0.13.0

5. Assumptions

Preconditions taken as true for the pipeline to run correctly:

- **Pre-cleaned Data:** The raw trajectory_snapshots.csv has already passed EDA and basic feature engineering.
 - **No residual missing values:** After preprocessing, no NaN values remain in the feature set.
 - **Snapshot consistency:** All temporal snapshots of a single customer share the same deal_id / company_id.
 - **Labels are correct:** Target labels (churn, renewal, etc.) have been correctly assigned during preprocessing.
 - **Static feature engineering:** Features are pre-engineered; the pipeline does not create new domain features.
 - **Single active experiment:** MLflow is treated as a local experiment tracker (mlruns/ folder).
-

6. Constraints

Inherent limitations of the current design:

- **Binary & Multiclass only:** Supports only classification tasks — regression-based churn scoring is not directly supported.
- **GridSearchCV — Exhaustive Search Only:** Tests every single combination. No Bayesian search. Example: 3 params × 4 values = 64 full model trainings.
- **Local MLflow tracking only:** MLflow tracking URI points to the local mlruns/ folder. No remote registry configured.
- **No real-time inference:** The pipeline is batch-oriented. Online/streaming inference is not supported.
- **Python 3.13+ required:** Backward compatibility with older Python versions is not guaranteed.
- **Autolog child run cap (5 runs visible):** MLflow autolog captures at most 5 child runs per GridSearch — even if 50+ combinations are evaluated.

Design Decision — Model Choice (Random Forest & XGBoost)

The pipeline intentionally limits model selection to Random Forest and XGBoost. These tree-based ensemble models require no feature scaling, handle mixed data types natively, are robust to outliers, and provide built-in feature importance. See README.md → Why Random Forest? for the full comparison.

7. Use Cases

Use Case	Description
Binary Churn Detection	Predict whether a customer will churn (Yes/No) and flag high-risk accounts for proactive outreach.
Multiclass Outcome Scoring	Classify customers into Churn / Renewal / Expansion / Downsell to prioritize the right intervention per segment.
Driver Analysis	Surface the Top 20 financial and engagement features driving churn so Customer Success can take targeted actions.
Experiment Tracking	Compare multiple model runs (RF vs XGBoost, Binary vs Multiclass) using the MLflow UI to select the champion model.
Model Registry & Versioning	Register versioned models (e.g. churn_binary_random_forest_v3) in MLflow for traceable, auditable deployment decisions.
Deployment-Ready Export	Serve the best estimator via mlflow models serve or load via mlflow.pyfunc.load_model() for batch scoring.

8. How to Use

Step 1 — Install Dependencies

```
pip install -r requirements.txt
```

Step 2 — Configure the Pipeline

Edit `src/config.py`:

```
MODE = ClassificationType.BINARY      # or MULTICLASS
MODEL = "random_forest"               # or "xgboost"
PRE_PROCESSING_FLAG = False           # Set True to force re-preprocessing
```

Step 3 — Run the Orchestrator Notebook

Open `churn_model_pipeline_mlflow.ipynb` and Run All Cells. This will:

- Preprocess raw data (if flagged)
- Load, split and verify data
- Build the preprocessing + model pipeline
- Tune hyperparameters via GridSearchCV
- Evaluate on the held-out test set
- Log everything to MLflow (params, metrics, plots, model)
- Register the best model in the MLflow Model Registry

Step 4 — Monitor Experiments in MLflow UI

```
mlflow ui
```

Open <http://127.0.0.1:5000> to compare run metrics, view feature importance charts, ROC curves, and inspect registered model versions.

Step 5 — Access Outputs

Output	Location
Trained model (.pkl)	model/ directory

Output	Location
MLflow run artifacts	mlruns/ directory
Classification report	MLflow → Artifacts tab → classification_report.txt
Feature importance chart	MLflow → Artifacts tab → feature_importance.png

Step 6 — Serve the Model (Optional)

```
mlflow models serve -m "runs://sklearn_model" --port 1234
```

Or load programmatically:

```
import mlflow.pyfunc
model = mlflow.pyfunc.load_model("runs://sklearn_model")
predictions = model.predict(X_new)
```